STAT340 Lecture 10: logistic regression

Brian Powers

Updated 11/18/2024

Introduction

These notes concern the problem of *logistic regression*.

This is a regression method that allows us to handle the situation where we have predictors and responses, but the response is binary rather than continuous.

Learning objectives

After this lecture, you will be able to

- Explain the motivation for logistic regression
- Fit a logistic regression model to data with one or more predictors and a binary response
- Interpret estimated coefficients in a logistic regression model

Logistic regression: motivation I

Consider the following data: the Pima Indian data set.

```
library(MASS)
head(Pima.te)
##   npreg glu bp skin  bmi   ped age type
## 1     6 148 72   35 33.6 0.627  50  Yes
## 2     1  85 66   29 26.6 0.351  31   No
## 3     1  89 66   23 28.1 0.167  21   No
## 4     3  78 50   32 31.0 0.248  26  Yes
## 5     2 197 70   45 30.5 0.158  53  Yes
## 6     5 166 72   19 25.8 0.587  51  Yes
```

These data were collected by the US National Institute of Diabetes and Digestive and Kidney Diseases. Each row of this data set corresponds to a woman of Pima Indian descent living near Phoenix Arizona. The columns include a number of biomarkers (e.g., glucose levels, blood pressure and age), as well as a column indicating whether or not each subject had diabetes (as measured according to measures laid out by the World Health Organization). See `?Pima.te` for more information.

Logistic regression: motivation II

Can we predict whether or not a given person has diabetes based on their biomarkers?

It is natural to cast this as a prediction problem just like the linear regression problems we have discussed in recent weeks: we are given pairs (X_i, Y_i), where X_i is a vector of predictors and Y_i is a response.

The difference is that now, Y_i is more naturally thought of as a binary label (in this case, indicating whether or not a subject has diabetes), instead of a continuous number.

Example: image classification

Suppose that we have a collection of images, and our goal is to detect whether or not a given image has a cat in it.

Our data takes the form of a collection of pairs (X_i, Y_i), $i = 1, 2, \ldots, n$ where X_i is an image (i.e., a rectangle of pixels) and

$$Y_i = \begin{cases} 1 & \text{if image } i \text{ contains a cat} \\ 0 & \text{if image } i \text{ does not contain a cat.} \end{cases}$$

Example: fraud detection

A credit card service might like to be able to detect whether or not a given transaction is fraudulent.

Predictors X_i might take the form of things like transaction amount, location and time, and the binary label Y_i corresponds to whether or not the transaction is fraudulent.

Example: brain imaging

In neuroscience a common task is to detect whether or not a subject has a disease or disorder (e.g., Parkinson's or schizophrenia) based on brain imaging data obtained via functional magnetic resonance imaging (fMRI).

Our labels Y_i are given by

$$Y_i = \begin{cases} 1 & \text{if subject } i \text{ has the disease/disorder} \\ 0 & \text{if subject } i \text{ does not have the disease/disorder.} \end{cases}$$

Our predictors X_i for subject $i \in \{1, 2, \ldots, n\}$ might consist of a collection of features derived from the brain imagining (e.g., average pixel values or correlations between different locations in the brain).

Our goal is to predict Y_i based on the observed imaging data (or image-derived features) X_i.

Formulating a model

A natural approach to this problem would be to just take our existing knowledge of linear regression and apply it here. If our predictor X_i has p dimensions,

$$X_i = (X_{i,1}, X_{i,2}, X_{i,3}, \ldots, X_{i,p})^T \in \mathbb{R}^p,$$

then we might try to fit a model of the form

$$Y_i = \beta_0 + \beta_1 X_{i,1} + \beta_2 X_{i,2} + \cdots + \beta_p X_{i,p}$$

The obvious problem with this is that the right-hand side of this equation is an arbitrary **real number**, whereas the left-hand side (i.e., Y_i) is 0 or 1.

Example: linear regression on the Pima data set I

For convenience let's add a column to the `Pima.te` data set.

The `type` column is `Yes` or `No` according to whether or not each subject has diabetes.

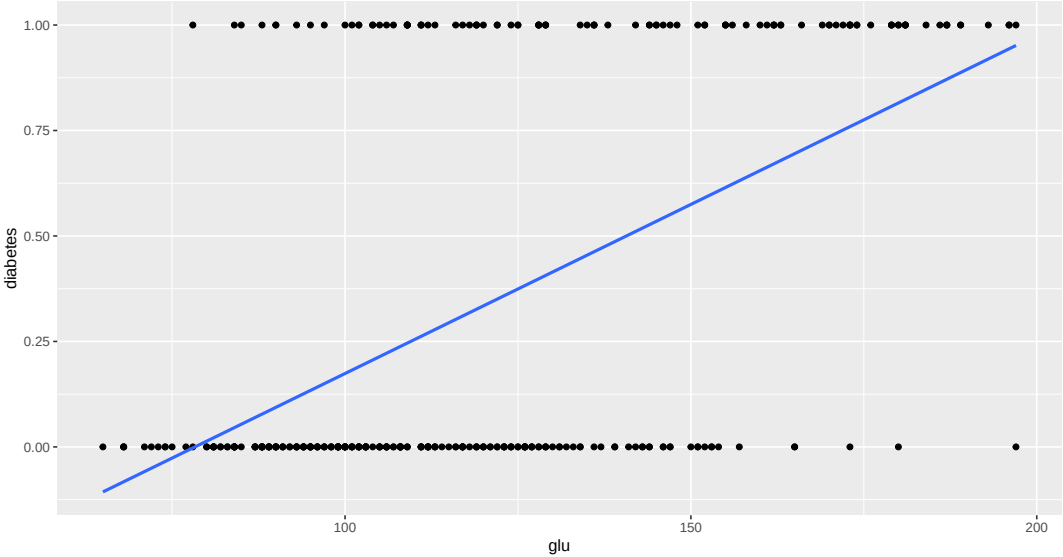
```
Pima.te$diabetes <- ifelse( Pima.te$type=='Yes', 1, 0)
head(Pima.te)
##   npreg glu bp skin  bmi   ped age type diabetes
## 1     6 148 72   35 33.6 0.627  50  Yes        1
## 2     1  85 66   29 26.6 0.351  31   No        0
## 3     1  89 66   23 28.1 0.167  21   No        0
## 4     3  78 50   32 31.0 0.248  26  Yes        1
## 5     2 197 70   45 30.5 0.158  53  Yes        1
## 6     5 166 72   19 25.8 0.587  51  Yes        1
```

Now, let's use `glu` as our predictor and fit a simple linear regression model.

Example: linear regression on the Pima data set II

```
##
## Call:
## lm(formula = diabetes ~ 1 + glu, data = Pima.te)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -0.9516 -0.2701 -0.1138  0.2408  1.0025
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.6278036  0.0892443  -7.035 1.16e-11 ***
## glu          0.0080171  0.0007251  11.057  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4023 on 330 degrees of freedom
## Multiple R-squared:  0.2703, Adjusted R-squared:  0.2681
## F-statistic: 122.3 on 1 and 330 DF,  p-value: < 2.2e-16
```

Example: linear regression on the `Pima` data set III



Example: linear regression on the `Pima` data set IV

The fitted model takes in a number (i.e., `glu`) and outputs a number that is our prediction for Y_i. The trouble is that our responses are binary, but our outputs can be anything. If our goal is to predict y, and we predict y to be $\hat{\beta}_0 + \hat{\beta}_1 x$, then we can (potentially) predict arbitrary large or small values for y, if x is suitably large or small.

We want our prediction to be 0 or 1.

We could do something like "clipping" our output $\hat{\beta}_0 + \hat{\beta}_1 x$ to whichever of 0 or 1 is closest or something like that.

But there is a better way.

An especially simple one is *logistic regression*, which seeks to keep the "linear combination of predictors" idea that we like so much from linear regression, but modify how we make predictions to be better-suited to the "binary response" setting.

Logistic regression I

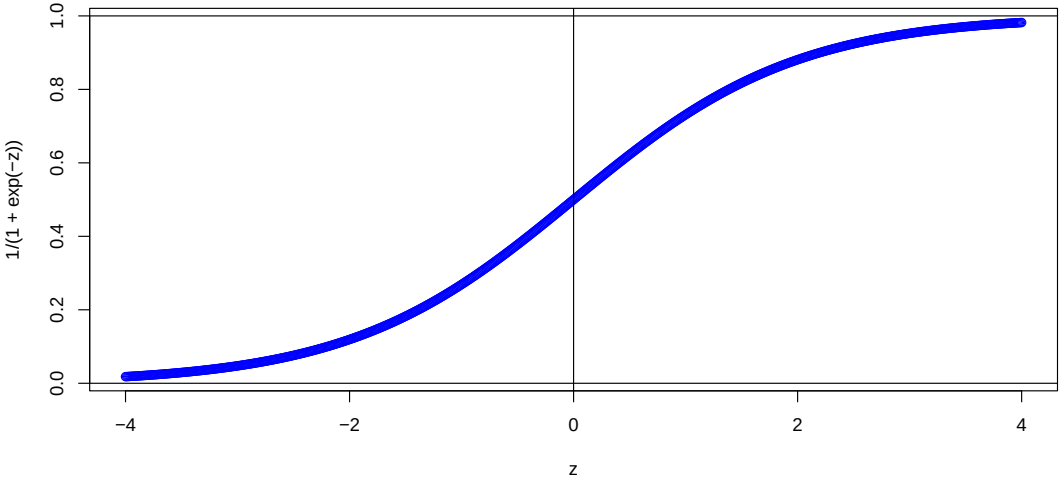
The core problem with applying linear regression to a binary response is that the output of linear regression can be any number, while we are really only interested in outputs in $\{0, 1\}$. How might we transform our *linear* prediction, $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$, into something more sensible?

Logistic regression solves this problem by taking our linear prediction $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$ and turning it into a *probability*, i.e., a number between 0 and 1. We do that using the *logistic function*,

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z}.$$

Logistic regression II

Here is a plot of the function:



The logistic function is an example of a sigmoid function ("S-shaped").

Logistic regression III

Using the logistic function, we can modify the output of our model by passing our linear regression model's prediction ($\in \mathbb{R}$) as the input of the logistic function so that it outputs a number between zero and one.

$$\sigma\left(\hat{y}\right) = \sigma\left(\hat{\beta}_0 + \hat{\beta}_1 x\right) = \frac{1}{1 + \exp\{-(\hat{\beta}_0 + \hat{\beta}_1 x)\}} = \frac{\exp\{\hat{\beta}_0 + \hat{\beta}_1 x\}}{1 + \exp\{\hat{\beta}_0 + \hat{\beta}_1 x\}}$$

Because this is between 0 and 1, we can interpret this as a probability:

$$\Pr[Y_i = 1; X_i = x, \beta_0, \beta_1] = \sigma\left(\beta_0 + \beta_1 x\right) = \frac{1}{1 + \exp\{-(\beta_0 + \beta_1 x)\}}$$

The semi-colon notation there is just to stress that we are treating β_0 and β_1 as parameters and the data x as an *input*, but *not* as things that we are conditioning on.

Logistic regression IV

Now, when it comes time to make a prediction on input x, we can pass $\hat{\beta}_0 + \hat{\beta}_1 x$ into the sigmoid function, and predict

$$\hat{y} = \begin{cases} 1 & \text{if } \sigma(\hat{\beta}_0 + \hat{\beta}_1 x) > \frac{1}{2} \\ 0 & \text{if } \sigma(\hat{\beta}_0 + \hat{\beta}_1 x) \leq \frac{1}{2}. \end{cases}$$

Logistic regression V

In the multivariate setting, where our i-th predictor X_i takes the form

$$X_i = (X_{i,1}, X_{i,2}, \ldots, X_{i,p}),$$

multiple logistic regression is the straight-forward extension of this idea. Given coefficients $\hat{\beta}_0, \hat{\beta}_1, \ldots, \hat{\beta}_p$, we output a probability

$$\begin{aligned}
\Pr[Y_i = 1; X_i = (x_1, x_2, \ldots, x_p), \hat{\beta}_0, \hat{\beta}_1, \ldots, \hat{\beta}_p] \\
&= \sigma\left(\hat{\beta}_0 + \hat{\beta}_1 x_1 + \cdots + \hat{\beta}_p x_p\right) \\
&= \frac{1}{1 + \exp\{-(\hat{\beta}_0 + \hat{\beta}_1 x_1 + \cdots + \hat{\beta}_p x_p)\}}
\end{aligned}$$

Logistic regression in R I

The function that we use in R is `glm`. "GLM" stands for *generalized* linear model. That is, we are generalizing linear regression. In particular, we are generalizing linear regression by doing linear regression, but then passing the linear regression prediction $\beta_0 + \beta_1 x$ through another function.

To perform *logistic* regression, we need to specify to R that we are using a "binomial" family of responses– our response Y is binary, so it can be thought of as a Binomial random variable, with $\Pr[Y = 1] = \sigma(\beta_0 + \beta_1 X)$.

Other than that, fitting a model with `glm` is basically the same as using plain old `lm`.

Logistic regression in R II

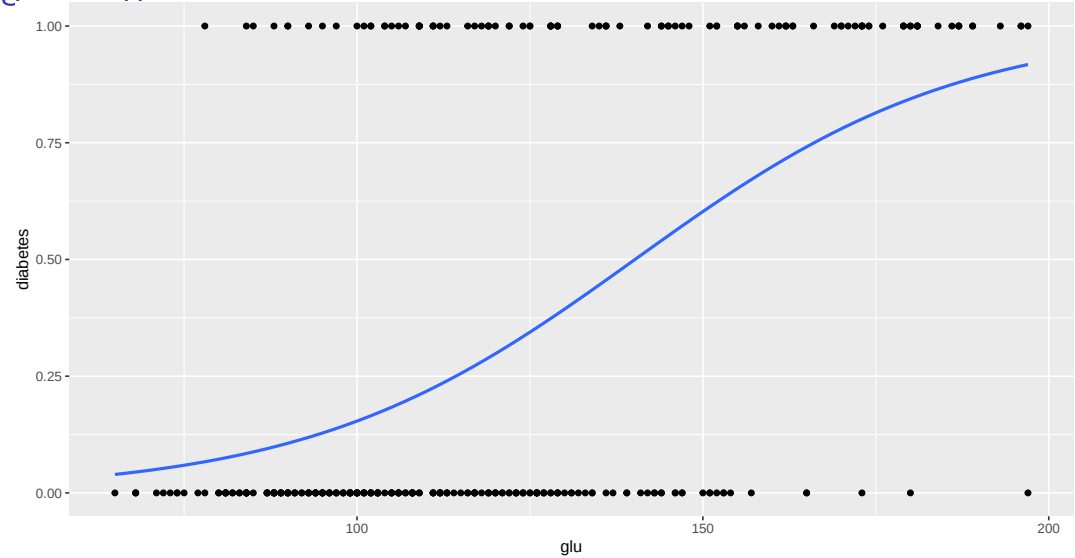
```
Call:
glm(formula = diabetes ~ 1 + glu, family = binomial, data = Pima.te)

Deviance Residuals:
    Min       1Q   Median       3Q      Max
-2.2343  -0.7270  -0.4985   0.6663   2.3268

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -5.946808   0.659839  -9.013   <2e-16 ***
glu          0.042421   0.005165   8.213   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

    Null deviance: 420.30  on 331  degrees of freedom
Residual deviance: 325.99  on 330  degrees of freedom
AIC: 329.99
```

Logistic regression in R III

But notice now that our model's outputs are always between 0 and 1:

```
pp <- ggplot( Pima.te, aes(x=glu, y=diabetes) ) + geom_point();
pp <- pp + geom_smooth(formula='y ~ 1+x', se=FALSE,
                       method='glm',
                       method.args=list(family = "binomial") );
# Note that geom_smooth needs an extra list() of arguments to specify the
# extra arguments that we passed to the glm() function above.
# In particular, we need to tell the GLM to use a binomial response.
pp
```

Logistic regression in R IV

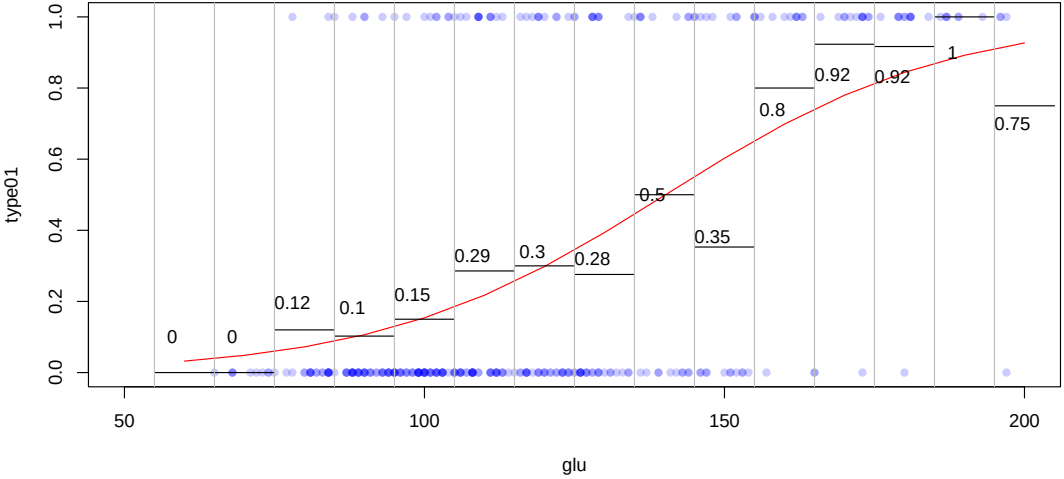


Logistic regression in R V

If we interpret our model's output as a probability, namely the probability that we observe label Y for predictors X, then it is clear that as `glu` increases, our model thinks it is more likely that a person has diabetes.

Logistic regression in R VI

When we plot the proportion along with the curve, it becomes more clear what the model is predicting.



Logistic regression in R VII

Just as with linear regression, our logistic model object will make predictions for new, previously unseen predictors if we use the `predict` function.

```
# Reminder: we pass a data frame into the predict function, and our model
# will produce an output for ease row of that data frame.
# So this is making a prediction for a subject with `glu=200`.
# Looking at our scatter plot above, it's clear that `glu=200` is at the
# far upper end of our data distribution, so we should expect our model to
# produce an output close to 1.
predict(pima_logistic, type='response', newdata = data.frame(glu=200) )
##         1
## 0.9267217
```

Interpreting model coefficients

In linear regression, our model coefficients had a simple interpretation: β_1 was the average change in response associated with a unit change in the corresponding predictor. Is there an analogous interpretation for logistic regression?

Well, there is, but it requires a bit of extra work to define some terms so that we can talk sensibly about *what* changes in response to a change in a predictor.

We need to first understand the idea of odds (and log-odds).

Odds vs Probability

The *odds* of an event E with probability p are given by

$$\text{Odds}(E) = \frac{p}{1-p}.$$

That is, the odds of an event is the ratio of the probability of the event happening to the probability that it doesn't happen.

So, for example, if we have an event that occurs with probability $1/2$, the odds of that event are $(1/2)/(1/2) = 1$, which we usually say as "one to one odds".

Interpreting model coefficients (cont.)

So, let's suppose that we have a logistic regression model with a single predictor, that takes predictor x and outputs a probability

$$p(x) = \frac{1}{1 + \exp\{-(\beta_0 + \beta_1 x)\}}.$$

and note that

$$1 - p(x) = \frac{\exp\{-(\beta_0 + \beta_1 x)\}}{1 + \exp\{-(\beta_0 + \beta_1 x)\}}$$

The *odds* associated with this probability are

$$\text{Odds}(x) = \frac{p(x)}{1 - p(x)} = \frac{1}{\exp\{-(\beta_0 + \beta_1 x)\}} = \exp\{\beta_0 + \beta_1 x\}.$$

Since the odds can get very large or very small, especially once we start working with small probabilities, it is often much easier to work with the *logarithm* of the odds, usually called the "log-odds" for short.

If we look at the *log* odds associated with our logistic regression model,

$$\text{Log-Odds}(x) = \log\left[\text{Odds}(x)\right] = \log\left[\exp\{\beta_0 + \beta_1 x\}\right] = \beta_0 + \beta_1 x.$$

Fitting the model I

Recall *ordinary least squares* regression, where we had the following *loss function*:

$$\ell(\beta_0, \beta_1) = \sum_{i=1}^{n} (y_i - (\beta_0 + \beta_1 x_i))^2 = \sum_{i=1}^{n} (\hat{y}_i - y_i)^2,$$

where $\hat{y}_i = \beta_0 + \beta_1 x_i$ is the predicted y_i based on the coefficients β_0, β_1.

This function measures how "wrong" our model is if we use coefficients β_0 and β_1, and we choose these coefficients in such a way as to make the loss function as small as possible.

Now that we're working with logistic regression instead of linear, how do we decide what makes a good choice of coefficient? The same loss function won't work well.

Fitting the model II

Our logistic regression model takes a predictor and outputs a probability that the response is 1. That is, for our i-th predictor, our model outputs:

$$\Pr[Y_i = 1; X_i = x, \beta_0, \beta_1] = \frac{1}{1 + \exp\{-(\beta_0 + \beta_1 x)\}}.$$

In other words, this is the probability that our model gives to the event that the i-th response is 1. On the other hand, our model assigns probability to the event that the i-th response is 0 given by

$$\begin{aligned}\Pr[Y_i = 0; X_i = x, \beta_0, \beta_1] &= 1 - \Pr[Y_i = 1; X_i = x, \beta_0, \beta_1] \\ &= \frac{\exp\{-(\beta_0 + \beta_1 x)\}}{1 + \exp\{-(\beta_0 + \beta_1 x)\}} \\ &= \frac{1}{1 + \exp\{\beta_0 + \beta_1 x\}}.\end{aligned}$$

Fitting the model III

Now, if the i-th response really is 0, then we want this number to be big. And if the i-th response really is 1, we want $1/(1+\exp\{-(\beta_0+\beta_1 x)\})$ to be big.

Further, we want this pattern to hold for all of our data. That is, we want our model to give a "high probability" to all of our data. If our i-th observation takes the form (x_i, y_i), then the probability of our data under the model, assuming the observations are independent, is

$$\prod_{i=1}^{n} \Pr[Y_i = y_i; X_i = x_i, \beta_0, \beta_1].$$

For one of our data points, using the fact that y_i is either 0 or 1, we can write

$$\Pr[Y_i = y_i; X_i = x_i, \beta_0, \beta_1] = \left(\frac{1}{1+\exp\{-(\beta_0+\beta_1 x)\}}\right)^{y_i} \left(\frac{1}{1+\exp\{\beta_0+\beta_1 x\}}\right)^{1-y_i}.$$

So the probability of our whole data set is

$$\prod_{i=1}^{n} \left(\frac{1}{1+\exp\{-(\beta_0+\beta_1 x)\}}\right)^{y_i} \left(\frac{1}{1+\exp\{\beta_0+\beta_1 x\}}\right)^{1-y_i}.$$

Fitting the model IV

We call this the *likelihood* of the data. Maximum likelihood model fitting means to pick our model coefficients so that this quantity is maximized.

How do we do that? Maximizing a sum is easier than maximizing a product, and because logarithms are monotonic transformations, we can more easily maximize the *log likelihood*. (Logs turn products into sums, and sums are easy to work with).

$$\log \prod_{i=1}^{n} \left(\frac{1}{1+\exp\{-(\beta_0+\beta_1 x)\}} \right)^{y_i} \left(\frac{1}{1+\exp\{\beta_0+\beta_1 x\}} \right)^{1-y_i}$$
$$= \sum_{i=1}^{n} \log \left(\frac{1}{1+\exp\{-(\beta_0+\beta_1 x)\}} \right)^{y_i} \left(\frac{1}{1+\exp\{\beta_0+\beta_1 x\}} \right)^{1-y_i}$$

This can be rewritten as

$$\sum_{i=1}^{n} y_i \log \left(\frac{1}{1+\exp\{-(\beta_0+\beta_1 x)\}} \right) + (1-y_i) \log \left(\frac{1}{1+\exp\{\beta_0+\beta_1 x\}} \right).$$

Fitting the model V

Logistic regression chooses the coefficients β_0 and β_1 to make this log-likelihood large.

Equivalently, we *minimize* the *negative* log likelihood,

$$\ell(\beta_0, \beta_1) = -\sum_{i=1}^{n} \left[y_i \log\left(\frac{1}{1 + \exp\{-(\beta_0 + \beta_1 x)\}} \right) + (1 - y_i) \log\left(\frac{1}{1 + \exp\{\beta_0 + \beta_1 x\}} \right) \right].$$

Our loss function for logistic regression is the *negative log likelihood*.

These optimized coefficients do not have nice closed form solutions. They are found in an iterative optimization algorithm Still, when we conduct logistic regression in R, this minimization problem is solved for us using tools from optimization.

Residual Deviance

A measure of model fit will see in the R output is the Residual deviance. It is exactly -2 times the log likelihood.

```
Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -5.946808   0.659839  -9.013   <2e-16 ***
glu          0.042421   0.005165   8.213   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

    Null deviance: 420.30  on 331  degrees of freedom
Residual deviance: 325.99  on 330  degrees of freedom
AIC: 329.99
```

Example: Maximum Likelihood Estimation of Bernoulli "p" parameter I

Suppose we have a Bernoulli r.v. $X \sim Bern(p)$, and we observe data $X_1, X_2, \ldots, X_n$ drawn independently from the distribution. We should choose our estimate to be the number that makes our data as "likely" as possible.

By independence,

$$\Pr[X_1 = x_1, X_2 = x_2, \ldots, X_n = x_n; p] = \prod_{x_i=1} p \prod_{x_i=0} (1-p)$$

Let's take the log of this, instead, to make it a sum:

$$\begin{aligned}\log \Pr[X_1 = x_1, X_2 = x_2, \ldots, X_n = x_n; p] &= \log \left(\prod_{x_i=1} p \prod_{x_i=0} (1-p) \right) \\ &= \sum_{x_i=1} \log p + \sum_{x_i=0} \log(1-p),\end{aligned}$$

where we made repeated use of the fact that the logarithm of a product is the sum of the logarithms.

Example: Maximum Likelihood Estimation of Bernoulli "p" parameter II

If we let $x$be the number of 1's observed, then $n - x$ is the number of 0's:

$$l(p) = x \ln p + (n - x) \ln(1 - p)$$

The maximum can be found by taking a derivative. Remember that p is the variable in this case.

$$\frac{d}{dp} l(p) = x \frac{1}{p} + (n - x) \frac{-1}{1 - p}$$

The maximum occurs when this equals zero.

$$\frac{x}{p} = \frac{n - x}{1 - p} \Rightarrow x - xp = np - xp \Rightarrow x = np$$

So the only critical point is $\hat{p} = \frac{x}{n}$, which is the sample proportion. So $\hat{P}$ is the maximum likelihood estimator, along with being the ordinary least squares estimator.

Model Assessment - the confusion Matrix

If we decide to make predictions that $Y = 1$ based on whether $\hat{y} \geq p$ then we can construct a matrix of outcomes

```
results <- data.frame(prediction =(predict(pima_logistic, type="response")>.5), trueLabel = Pima.te$t
table(results)
##           trueLabel
## prediction  No Yes
##      FALSE 200  53
##      TRUE   23  56
```

In this table the rows correspond with the decision "Do we predict $y = 1$?"
"FALSE" indicates that $\hat{y} < .5$ and *TRUE* indicates $\hat{y} \geq .5$. Columns refer to the true value of y_i. By thinking of the binary classification problem as a hypothesis test $H_0 : y = 0$ vs. $H_A : y = 1$, we can consider each of these four outcomes in the following way:

```
      No Yes
FALSE TN  FN
TRUE  FP  TP
```

Logistic Model Diagnostics

Accuracy: $\dfrac{TP + TN}{n}$

```
(200+56)/(200+53+23+56)
## [1] 0.7710843
```

True positive rate (Sensitivity, power): $P(reject|H_A) = P(TRUE|Yes) = \dfrac{TP}{TP + FN}$

```
56/(56+53)
## [1] 0.5137615
```

False positive rate (Type 1 error rate): $P(reject|H_0) = P(TRUE|No) = \dfrac{FP}{TN + FP}$

```
23/(200+23)
## [1] 0.103139
```

Logistic Model Diagnostics (more)

Precision (Positive Predictive Value) : $P(TP|Positive) = \dfrac{TP}{TP + FP}$

```
23/(23+56)
```

Modifying the threshold

The confusion matrix will change depending on the threshold we use to predict $y = 1$.

For example, if we use $p = 0$ (i.e. if $\hat{y} \geq 0$ we predict 1) then we will get

```
table(factor(predict(pima_logistic, type="response")>=0,c(FALSE,TRUE)), Pima.te$type)
##
##          No Yes
##   FALSE   0   0
##   TRUE  223 109
```

FPR $= 1$ and TPR $= 1$.

On the other extreme if we use $p = 1$ (i.e. if $\hat{y} \geq 1$ we predict 1) we will get

```
table(factor(predict(pima_logistic, type="response")>=1,c(FALSE,TRUE)), Pima.te$type)
##
##          No Yes
##   FALSE 223 109
##   TRUE    0   0
```

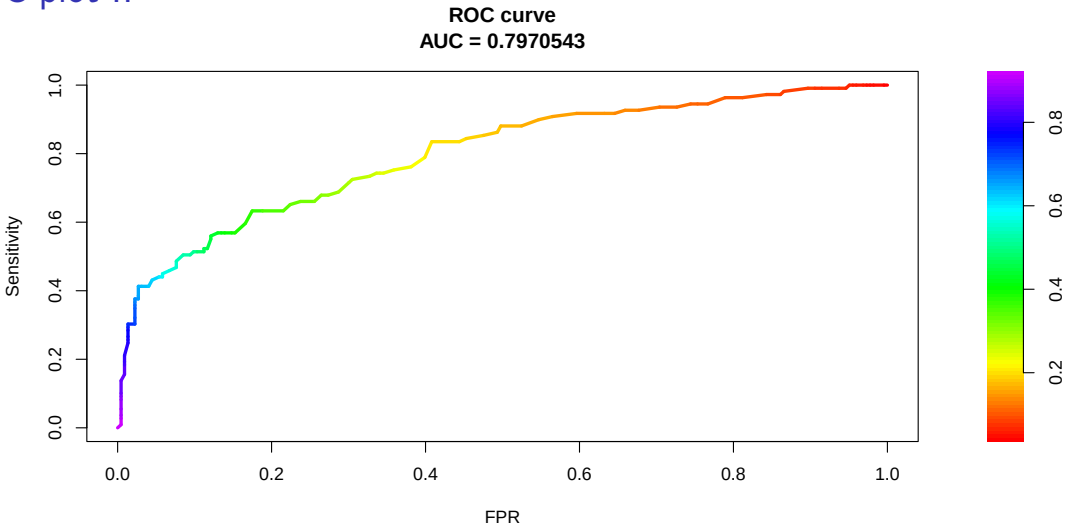
With FPR=0 and TPR=0.

ROC plot I

As we slide the threshold from 0 to 1, the model will achieve different values for FPR and TPR. These can be visualized in a Receiver Operating Characteristic (ROC) plot

```
#install.packages("PRROC")
library(PRROC)
## Loading required package: rlang
PRROC_obj <- roc.curve(scores.class0 = predict(pima_logistic, type="response"),
                       weights.class0=Pima.te$typeTF,
                       curve=TRUE)
plot(PRROC_obj)
```

ROC plot II



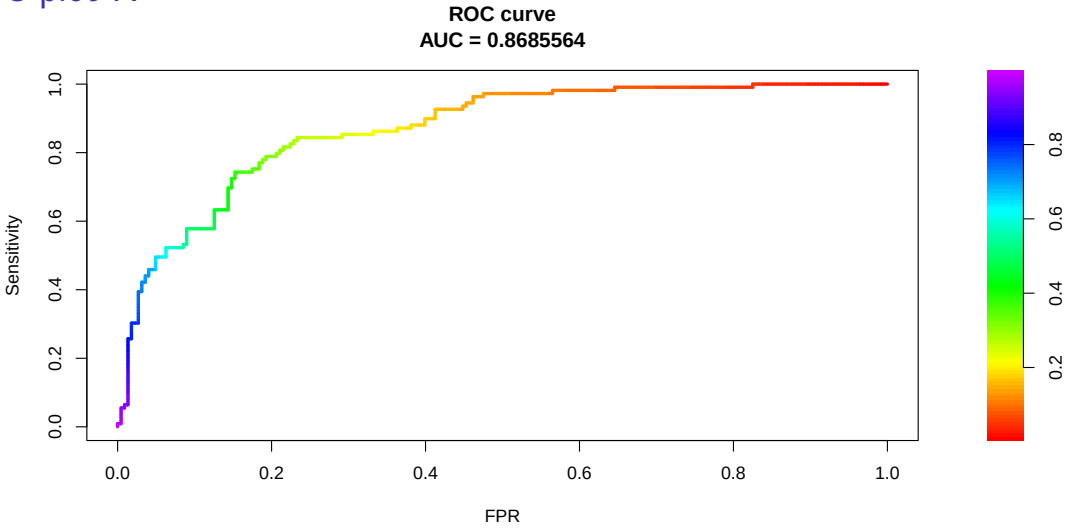
ROC plot III

The AUC stands for "area under curve" which can be as low as .5 for the null model and as high as 1 for a perfect classifier. The higher the AUC the better.

Consider a full model where we include all predictors

```
pima_logistic.full <- glm(diabetes ~ npreg+glu+bp+skin+bmi+ped+age, data=Pima.te, family=binomial)
PRROC_obj2 <- roc.curve(scores.class0 = predict(pima_logistic.full, type="response"),
                        weights.class0=Pima.te$typeTF,
                        curve=TRUE)
plot(PRROC_obj2)
```

ROC plot IV



The Null Logistic Model I

```
##
## Call:
## glm(formula = diabetes ~ 1, family = binomial, data = Pima.te)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -0.7158     0.1169  -6.125 9.07e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 420.3  on 331  degrees of freedom
## Residual deviance: 420.3  on 331  degrees of freedom
## AIC: 422.3
##
## Number of Fisher Scoring iterations: 4
```

The Null Logistic Model II

Notice that in the null model that the null deviance = residual deviance. So now we know what "Null deviance" is - it is the residual deviance for the null model. The null model will predict the same probability regardless of x

```
predict(pima_logistic.null, type="response")[1:10]
##         1         2         3         4         5         6         7         8
## 0.3283133 0.3283133 0.3283133 0.3283133 0.3283133 0.3283133 0.3283133 0.3283133
##         9        10
## 0.3283133 0.3283133
```

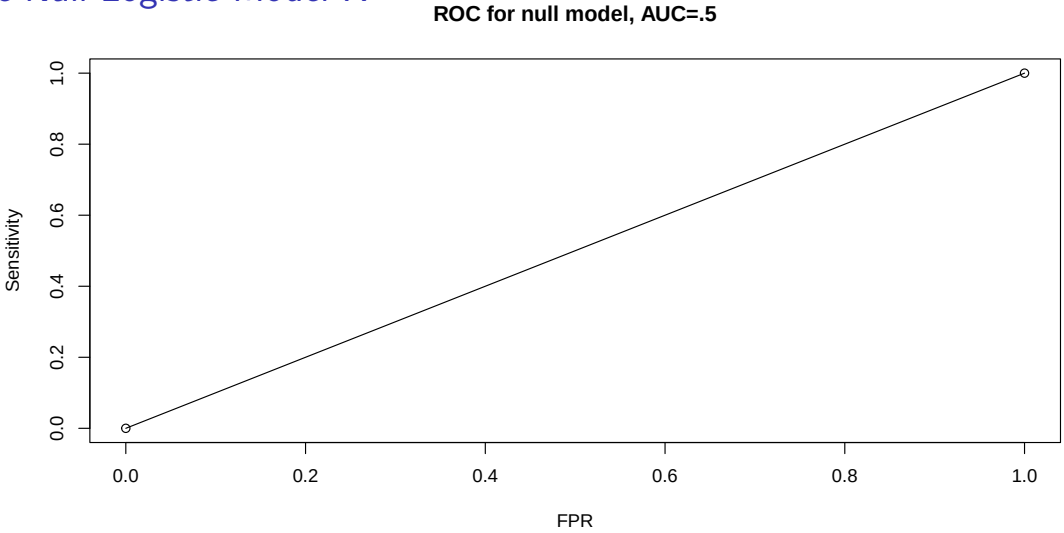
This predicted probability is equal to the proportion of 1s in the dataset

```
sum(Pima.te$diabetes==1)/nrow(Pima.te)
## [1] 0.3283133
```

The Null Logistic Model III

In terms of prediction, the null model will predict 1 or 0 for EVERY individual, so it either achieves TPR=FPR=1 or TPR=FPR=0 depending on the threshold. The ROC plot shows a straight line, but this is actually just connecting two points.

The Null Logistic Model IV



Review

- Relationship between probability, odds and log-odds
- Sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$
- Simple logistic model log-odds$(Y = 1) = \beta_0 + \beta_1 X_1$
- Predicting y from a logistic model
- interpreting the coefficients of a logistic model
- Z tests of significance of logistic coefficients
- fitting a logistic model (maximize log-likelihood / minimize residual deviance)
- likelihood / log likelihood / residual deviance
- confusion matrix for a logistic model, prediction statistics
- ROC curve
- null model